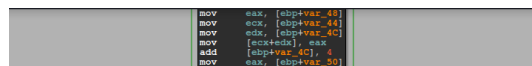
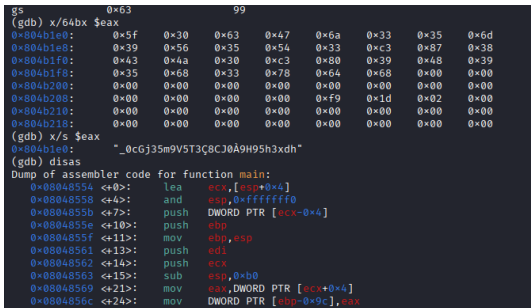


lundi 15 juin 2026 16:41





la variable var_94 contient cette valeur : "_OcGj35m9V5T3Ç8CJ0À9H95h3xdh"

```

00000910: .rodata:000489F0  const char 'format'
00000911: .rodata:000489F0  format
00000912: .rodata:000489F0  align 10h
00000913: .rodata:00048910  unk_0048910
00000914: .rodata:00048921  30h
00000915: .rodata:00048912  43h
00000916: .rodata:00048913  47h
00000917: .rodata:00048914  63h
00000918: .rodata:00048915  13h
00000919: .rodata:00048916  35h
00000920: .rodata:00048917  80h
00000921: .rodata:00048918  39h
00000922: .rodata:00048919  56h
00000923: .rodata:0004891A  35h
00000924: .rodata:0004891B  54h
00000925: .rodata:0004891C  33h
00000926: .rodata:0004891D  0C3h
00000927: .rodata:0004891E  87h
00000928: .rodata:0004891F  36h
00000929: .rodata:00048920  43h
00000930: .rodata:00048921  4Ah
00000931: .rodata:00048922  0C3h
00000932: .rodata:00048923  80h
00000933: .rodata:00048924  39h
00000934: .rodata:00048925  48h
00000935: .rodata:00048926  48h
00000936: .rodata:00048927  39h
00000937: .rodata:00048928  50h
00000938: 00000915 00048915: .rodata:00048915

```

```

0000048800 format      db '(%)-Syntax: %s [password] ',0Ah,0
0000048800             ; DATA XREF= mail
000004880E             align 10h
0000048900 unk_8048910  db 5Fh ; DATA XREF= mail
0000048911             db 30h ; 0
0000048912             db 63h ; 0
0000048913             db 47h ; G
0000048914             db 6Ah ; C
0000048915             db 33h ; 0
0000048916             db 35h ; 5
0000048917             db 6Dh ; m
0000048918             db 39h ; 0
0000048919             db 56h ; V
000004891A             db 35h ; 5
000004891B             db 54h ; 0
000004891C             db 33h ; 3
000004891D             db 0C3h
000004891E             db 87h ; 0
000004891F             db 36h ; 8
0000048920             db 43h ; C
0000048921             db 4Ah ; 0
0000048922             db 30h ; 0
0000048923             db 0C3h
0000048924             db 80h ; 0
0000048925             db 39h ; 9
0000048926             db 46h ; H
0000048927             db 35h ; 5
0000048928             db 35h ; 5
00000926 08048926 :.rodata:08048926

```

8/23/26, 01:20

```

0x0804869a <+326>: mov     DWORD PTR [esp+0x4],eax
0x0804869e <+330>: lea     eax,[ebp-0x2a]
0x080486a1 <+333>: mov     DWORD PTR [esp],eax
--Type <RET> for more, q to quit, c to continue without paging--
0x080486a4 <+336>: call    edx
0x080486a6 <+338>: mov     edi,DWORD PTR [ebp-0xc]
0x080486a9 <+341>: xor     edx,DWORD PTR [0:0x14]
0x080486b0 <+348>: je      0x080486b7 <main+355>
0x080486b2 <+350>: call    0x0804845c <__stack_chk_fail@plt>
0x080486b7 <+355>: add     esp,0xb0
0x080486bd <+361>: pop     ecx
0x080486be <+362>: pop     edi
0x080486bf <+363>: pop     ebp
0x080486c0 <+364>: lea     esp,[ecx-0x4]
0x080486c3 <+367>: ret

End of assembler dump.
(gdb) x/s $eax
0x0804b1f6: "H95h3xdh"
(gdb)

```

Après une nuit de sommeil voici ce que j'ai découvert en utilisant IDA et GDB :

The screenshot shows the IDA Pro interface with the assembly code for the function 'loc_004001C'. The code is written in x86 assembly with comments in French. The comments explain the logic of the function, including calculating the address of a buffer, copying data, and checking for a null terminator. A GDB console window is also visible, showing the execution of the program and the value of the 'eax' register.

The screenshot shows the IDA Pro interface with the assembly code for the function 'loc_004001C'. The code is written in x86 assembly with comments in French. The comments explain the logic of the function, including calculating the address of a buffer, copying data, and checking for a null terminator. A GDB console window is also visible, showing the execution of the program and the value of the 'eax' register.

On remarque bien que la chaîne comparé avec notre input est la suivante :
 _0cGjc5m_5v\nc8CJOA9

Ce qu'on peut faire c'est la passer en argument directement sauf qu'il y a des chaînes de caractères qui ne sont pas imprimables comme le '\n' et le '\r' donc on doit le convertir en bytes (hex) ce qui nous donne : _0cGjc5m_5v\nc8CJOA9

./crackme %s(printf "_0cGjc5m_5v\nc8CJOA9")

Et on a ce maudit password que j'avais trouvé avant dans la fonction blowfish
 Qui était libéré!

Avec ltrace on peut voir ce que retourne le binaire :

ltrace ./crackme PASSWORD

```

[adelchat@adelchat-20251204 crackk]$ ltrace ./crackme AAAAAA
__libc_start_main([ "./crackme", "AAAAAA" ] <unfinished ...>
malloc(29)                                = 0x8c121a0
memcpy(0x8c121a0, "_OcGjc5m_5\r\n\303\2078CJ0\303\2009H9Sh3xdh(0", 31) = 0x8c121a0
memcpy(0xfffff4f3a, "Celebration0", 13)    = 0xfffff4f3a
strcpy(0xfffff4f3e, "AAAAAA")              = 0xfffff4f3e
puts("\303\251rification de votre mot de pa...Vérification de votre mot de passe..
)                                           = 38
strcmp("AAAAAA", "_OcGjc5m_5\r\n\303\2078CJ0\303\2009") = -1
printf("Le vote de la sagesse te guidera"...le vote de la sagesse te guidera, tache de trouver le mot de passe petit padawaan
)                                           = 84
exit(0 <no return ...>
+++ exited (status 0) +++
[adelchat@adelchat-20251204 crackk]$

```

On peut voir ici la chaîne comparée directement... pas besoin de faire toute mon analyse du programme. Mais je suis quand même contente de l'avoir fait car j'ai appris des trucs.

On peut hijacker une librairie apparemment :

Si on utilise file on peut voir si le binaire utilise une librairie dynamique ou pas :

```

On voit qu'il fait un strcmp, Regardons avec un file si il utilise les librairies dynamiques

$ file crackme
crackme: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), dynamically linked (uses shared libs), for GNU/Linux 2.6.8, not stripped

C'est le cas, on va utiliser un hijacking via LD_PRELOAD.

On écrit une fake librairie dont voilà le code source :

1. #include <stdio.h>
2. #include <string.h>
3. int strcmp(const char *s1, const char *s2)
4. {
5.     printf("argument1: %s, argument2: %s\n", s1, s2);
6.     return 0;
7. }

On la compile avec cette commande :

1. $ gcc strcmp.c -m32 -shared -fPIC -o /tmp/strcmp.so

Ne pas oublier le -m32 si on est en 64 bits, les binaires sont compilés en 32
Puis on set la variable LD_PRELOAD

```

On la compile avec cette commande :

```
1. $ gcc strcmp.c -m32 -shared -fPIC -o /tmp/strcmp.so
```

Ne pas oublier le -m32 si on est en 64 bits, les binaires sont compilés en 32

Puis on set la variable LD_PRELOAD

```
1. $ export LD_PRELOAD="/tmp/strcmp.so"
```

Et on lance le binaire :

```

1. $ ./crackme 13
2. Vérification de votre mot de passe..
3. argument1: 13, argument2: _OcGjc5m_.5
4. Ç8CJ0À9
5. '+' Authentication réussie...
6. U'r root! <br
7. sh 3.0 # password: liberté!

```

Bingo !

Cette technique mérite **absolument un article dédié** sur ton blog. Tu pourrais écrire :

"Le hijacking LD_PRELOAD : casser un crackme sans le reverser"

Structure suggérée :

1. Présentation du problème (un crackme avec strcmp)
2. Explication de LD_PRELOAD et du linker dynamique
3. Le code de la fake lib
4. La résolution en 5 commandes
5. Pourquoi/quand ça marche
6. Autres applications (debug, modding, malware)
7. Bonus : comment se protéger de ça (statique, syscalls directs, vérification d'intégrité)

C'est exactement le genre de **technique pratique et concrète** que les gens adorent lire en reverse engineering.

Question pour aller plus loin

Tu vois pourquoi le programme tente de cacher le mot de passe "liberté" à la fin avec la fonction blowfish ?

Et tu vois pourquoi les transformations dans main (le 'c' à la position 5, etc.) sont en fait des **leures** ? Avec LD_PRELOAD, on n'a même pas besoin de comprendre ces transformations !

Récap final pour ton blog :

Mot de passe attendu = "_OcGjc5m_5\r\nÇ8CJ0À9"

Mot de passe révélé par le programme = "liberté" (le drapeau du challenge)

Avec GDB pour changer une valeur :

```
set [char]@adresse-de-la-valeur=@nouvelle-valeur
```

Pour résoudre ce challenge, j'ai utilisé entièrement GDB :

```
$ gdb ./crackme
(gdb) disassemble main
```

On remarque l'appel de `%edx`

```
0x080486a4 <+336>:      call    *%edx
```

On va dans un premier temps faire un break sur le call :

```
(gdb) break *0x080486a4
```

on lance le programme

```
(gdb) run abcd
```

on a notre registre edx qui contient maintenant une valeur
on a fait un break sur l'adresse d'edx :

```
(gdb) break *($edx)
Breakpoint 2 at 0x080486c4
(gdb) continue
Continuing.
Breakpoint 2, 0x080486c4 in WPA ()
```

On arrive dans la fonction WPA, on l'a désassemble

```
(gdb) disassemble
```

les lignes qui nous intéressent sont :

```
0x080486fa <+54>:      test    %eax,%eax
0x080486fc <+56>:      jne     0x0804870f <WPA+75>
```

On peut voir il teste une valeur et que si, elle est pas bonne il saute une bonne partie de la fonction...

On va remplacer le jne, par un je

```
(gdb) set {char}0x080486fc=0x74
(gdb) continue
```

Et Bingo !

```
Continuing.
Vérification de votre mot de passe..
*) Authentification réussie...
U'r root!
```

```
sh 3.0 # password: liberté!
```